

OSDBMS-V-2026-T038

API SPECIFICATION

WALnut

Embedded REST API & Live Telemetry (SSE) Reference

Team: The Acid Mechanics – T038

Vinayak Suyal · Sushmit Singh Rawat · Kanak Rawat · Suhavi Jugran

Version 1.0 · Companion to the WALnut PRD & Architecture Document
Status: Draft for Review

Table of Contents

- 1. Introduction
- 2. Conventions
- 3. Shared Data Types
- 4. Error Model
- 5. Endpoints
- 6. Live Telemetry Stream (SSE)
- 7. End-to-End Example: Deadlock Scenario
- 8. Versioning & Change Management

1. Introduction

This document is the contract between the WALnut engine and any client of it — the React dashboard, load-test scripts, or manual testing tools such as curl/Postman. It specifies every HTTP endpoint exposed by the embedded REST server (cpp-httpplib), their request/response schemas, error behavior, and the Server-Sent Events (SSE) payloads that drive the dashboard's live views.

It formalizes Section 11 (API & Communication Layer) of the WALnut Architecture Document. Functional behavior of each underlying operation (validation rules, transaction semantics) is defined in the PRD; this document defines only the wire format.

2. Conventions

Aspect	Convention
Base URL	http://localhost:8080 (single-process, single-machine; port configurable at startup)
Content type	application/json for all request and response bodies unless noted otherwise
ID format	user_id, item_id are caller-supplied strings; bid_id is server-assigned, monotonically increasing
Timestamps	ISO 8601 UTC, e.g. 2026-09-09T14:03:21Z
Authentication	None. Per PRD Section 4.3, identity is a fixed user_id supplied by the caller — no login/token flow.
Idempotency	None of the fixed operations are idempotent; retries (including automatic deadlock-victim retries) issue a new call.

3. Shared Data Types

3.1 User

```
{
  "user_id": "string",
  "username": "string"
}
```

3.2 Item

```
{
  "item_id": "string",
  "title": "string",
  "starting_price": number,
  "status": "OPEN" | "CLOSED",
  "current_bid": number,
  "current_winner": "string | null"
}
```

3.3 Bid

```
{
  "bid_id": number,
  "item_id": "string",
  "user_id": "string",
  "amount": number,
  "bid_time": "ISO-8601 string"
}
```

4. Error Model

Every non-2xx response returns a JSON body of the following shape:

```
{
  "error": {
    "code": "string",
    "message": "string"
  }
}
```

HTTP Status	error.code	Meaning
400	VALIDATION_ERROR	Malformed request, e.g. missing field or non-numeric amount.
404	NOT_FOUND	Referenced user_id or item_id does not exist.
409	BID_TOO_LOW	placeBid amount is not strictly greater than the item's current_bid.
409	AUCTION_CLOSED	placeBid or reserveBundle attempted on an item whose status is CLOSED.
409	TRANSACTION_ABORTED	Transaction was chosen as a deadlock victim and aborted; the engine has already scheduled an automatic retry (Architecture Doc §9.2). The client may also retry manually.
500	INTERNAL_ERROR	Unexpected engine failure (e.g. storage I/O error).

5. Endpoints

POST /users

Maps to createUser. Registers a new user.

Request body

```
{
  "user_id": "string",
  "username": "string"
}
```

Response 201

```
{
  "user_id": "string",
  "username": "string"
}
```

Errors: 400 *VALIDATION_ERROR* (missing fields).

POST /items

Maps to `createItem`. Registers a new auction item with status `OPEN` and `current_bid = starting_price`.

Request body

```
{
  "item_id": "string",
  "title": "string",
  "starting_price": number
}
```

Response 201 — Item

Errors: 400 *VALIDATION_ERROR*.

GET /items

Convenience read endpoint (not a fixed engine operation in its own right) returning all items, used by the dashboard's bidding view to populate the active-items list.

Response 200

```
{
  "items": [ Item, ... ]
}
```

POST /items/{item_id}/bids

Maps to `placeBid(item_id, user_id, amount)`. This is the engine's central transaction (Architecture Doc §8.3). Reachable identically from the dashboard's manual “Place Bid” control and from load-test scripts — no separate demo path (PRD FR-3 / FR-11).

Request body

```
{
  "user_id": "string",
  "amount": number
}
```

Response 201 — Bid

Errors: 400 *VALIDATION_ERROR*, 404 *NOT_FOUND* (item or user), 409 *BID_TOO_LOW*, 409 *AUCTION_CLOSED*, 409 *TRANSACTION_ABORTED*.

GET /items/{item_id}/highest-bid

Maps to `getHighestBid(item_id)`.

Response 200

```
{
  "item_id": "string",
  "current_bid": number,
  "current_winner": "string | null"
}
```

Errors: 404 *NOT_FOUND*.

GET /items/{item_id}/bids

Maps to `listBids(item_id)`. Returns full bid history, most recent first.

Response 200

```
{
  "item_id": "string",
  "bids": [ Bid, ... ]
}
```

Errors: 404 *NOT_FOUND*.

POST /items/{item_id}/close

Maps to `closeAuction(item_id)`. Sets status to *CLOSED*; subsequent `placeBid` calls on this item return 409 *AUCTION_CLOSED*.

Response 200 — Item

Errors: 404 *NOT_FOUND*.

POST /reserve-bundle

Maps to `reserveBundle([item_A, item_B], user_id)`. A deliberately constructed multi-lock, all-or-nothing operation used to force and demonstrate a circular wait (PRD FR-7; Architecture Doc §9.3) — not part of normal single-item bidding.

Request body

```
{
  "item_ids": ["string", "string"],
  "user_id": "string"
}
```

Response 200

```
{
  "reserved": true,
  "item_ids": ["string", "string"]
}
```

Errors: 400 `VALIDATION_ERROR`, 404 `NOT_FOUND`, 409 `TRANSACTION_ABORTED` (this transaction was the deadlock victim — the engine auto-retries; see Architecture Doc §9.2).

6. Live Telemetry Stream (SSE)

GET /stream

A long-lived Server-Sent Events connection. Each event has an event: name and a JSON data: payload, consumed by the dashboard's systems-monitor view (Architecture Doc §11.2).

6.1 Event Types

event:	data: payload	Drives
lock.acquired	{ txn_id, item_id, mode }	Live lock table
lock.waiting	{ txn_id, item_id, waiting_on_txn_id }	Live lock table + wait-for graph edge
lock.released	{ txn_id, item_id }	Live lock table
deadlock.detected	{ cycle: [txn_id, ...] }	Wait-for graph highlight
deadlock.resolved	{ victim_txn_id, retried: true }	Wait-for graph update
wal.write	{ lsn, txn_id, type }	WAL activity view
checkpoint.complete	{ lsn }	WAL activity view
buffer.hit	{ page_id }	Buffer-pool hit/miss rate
buffer.miss	{ page_id }	Buffer-pool hit/miss rate

6.2 Example Frame

```
event: deadlock.detected
data: {"cycle": ["txn-17", "txn-22"]}
```

7. End-to-End Example: Deadlock Scenario

This sequence reproduces the reserveBundle deadlock demo referenced throughout the PRD and Architecture Document, as a concrete API call sequence.

- 1. POST /items (item_id: A), POST /items (item_id: B) — create both items.

- 2. Client T1: POST /reserve-bundle { item_ids: [A, B], user_id: u1 } — acquires lock on A, then blocks requesting B.
- 3. Client T2 (concurrently): POST /reserve-bundle { item_ids: [B, A], user_id: u2 } — acquires lock on B, then blocks requesting A.
- 4. The dashboard's /stream connection emits lock.waiting for both transactions, then deadlock.detected with the two-transaction cycle.
- 5. The engine aborts one transaction (deadlock.resolved), releases its locks, and automatically retries it.
- 6. The surviving transaction's /reserve-bundle call completes with 200 { reserved: true }; the retried one completes shortly after once the first releases its locks.

8. Versioning & Change Management

The API is unversioned for the duration of the eight-week project (single deployment target, no external consumers). Any breaking change to a request/response shape must be reflected in this document and communicated to all four team members before the owning subsystem's next integration point, per the milestone plan in the PRD.